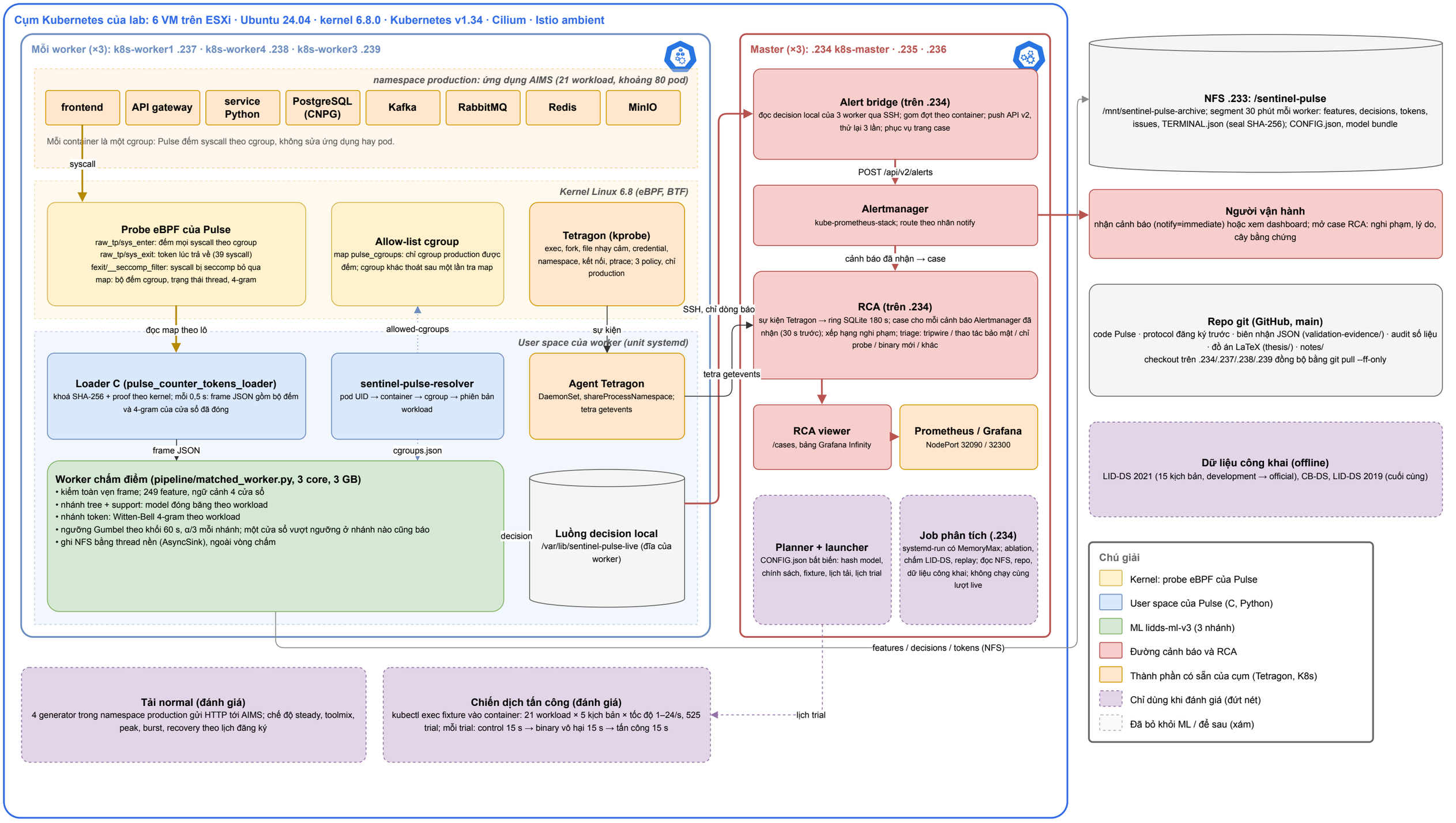
Sentinel Pulse: kiến trúc tổng thể (07/11/2026)

Phát hiện runtime trên Kubernetes từ syscall đếm trong kernel: ML lidds-ml-v3 (token + tree + support), cảnh báo qua Alertmanager, RCA từ sự kiện Tetragon. Khung tím đứt nét chỉ dùng khi đánh giá.



Đường dữ liệu chính: syscall của container → probe eBPF đếm theo cgroup và tạo token → loader đọc map mỗi 0,5 s → worker dựng feature, chấm 3 nhánh lidds-ml-v3 → decision ghi local và NFS → bridge gom đợt, đẩy vào Alertmanager → RCA dựng case cho đúng cảnh báo đó từ sự kiện Tetragon.
Mục tiêu: recall ≥ 95%, precision ≥ 95%, p95 kernel → cảnh báo ≤ 3 s; gánh cảnh báo và binary vô hại bị gắn cờ càng thấp càng tốt. Tripwire A1 và chiết khấu theo replica tắt từ 01/11 (luật thêm sau khi ML chốt).

Probe eBPF: pulse_counter_tokens (make tokens)

Matched full-ID observable + token select1f của lidds-ml-v3. Không còn ftrace_ops nào (amendment 1–2 của token-probe-overhead-v1): chỉ raw tracepoint và fexit/__seccomp_filter có sẵn từ probe trước.

Một syscall của process trong container được giám sát

syscall(nr, args)
x86_64; mọi thread

seccomp của container
SECCOMP_RET_ERRNO → syscall không chạy

cho phép

thân syscall (__x64_sys_*)
ret ≥ 0 hoặc −errno

trở về user space
syscall_exit_work

bỏ qua

Chương trình BPF (kernel 6.8, CO-RE, BTF)

raw_tp/sys_enter → pulse_record_call
1. cgroup_id → tra pulse_cgroups (miss: thoát ngay)
2. full_counts[nr]++ (0–1023)
3. bin cặp syscall (cùng thread, ≤ 5 s)++
4. tra hoặc tạo trạng thái thread (pulse_last_syscall, khoá pid_tgid)
5. syscall không trong bảng hook: phát token tên ngay
   syscall có hook: tok_pending = 1

fexit/__seccomp_filter
syscall bị seccomp bỏ qua (result −1, không phải recheck) không tới sys_enter: đếm như sys_enter (seccomp_skipped_or_emulated++), token tên lúc vào, tok_pending = 0 nên sys_exit không phát lần hai

raw_tp/sys_exit → pulse_tok_sys_exit
chỉ khi tok_pending (cùng thread, cùng cgroup); nr = syscall_id đã ghi ở sys_enter. Token theo bảng 39 syscall (pulse_token_ids.h):
• lỗi, syscall thuộc nhóm ERR (26 + futex): tên + errno
• đọc/ghi thành công, ret > 0: tên + min(15, ⌊log2 ret⌋)
• open, openat, openat2, creat thành công: tên + chế độ (w nếu O_WRONLY | O_RDWR | O_CREAT | O_TRUNC | O_APPEND) + thư mục đầu
• còn lại: tên
Thư mục đầu: fd trả về → struct file → đi ngược d_parent, vượt mount (bpf_loop ≤ 64 bước) tới gốc của task (gốc container) → FNV-1a của thành phần đầu, giữ 20 bit

.rodata.tokcfg
loader đặt trước khi nạp; verifier cắt nhánh tắt:
• futex_at_exit: futex là syscall có hook
• count_grams: cập nhật map 4-gram
B = exit-nofutex, C = exit, D = grams (điều kiện đo overhead; lượt live dùng D)

--token-exit fexit: 39 trampoline fexit thay sys_exit (chỉ để so sánh)

token lúc vào

token lúc vào

token lúc ra

pulse_tok_emit (gọi từ cả ba hook)
gram = (tok[0], tok[1], tok[2], token mới); khoá = (cgroup, cửa sổ = ktime / 0,5 s, gram); có trong map thì cộng nguyên tử, chưa có thì chèn BPF_NOEXIST (thua tranh chấp: tra lại); rồi dịch lịch sử của thread

lịch sử

đếm gram

pulse_cgroups
PERCPU_HASH, 1024 cgroup: full_counts[1024], 64 bin cặp, seccomp_skipped (allow-list)

pulse_last_syscall
LRU_HASH 131.072 thread: cgroup, seen_ns, syscall_id, tok[3], tok_pending

pulse_grams
HASH 262.144 entry; khoá 32 byte (cgroup, cửa sổ, 4 token); giá trị: số lần

pulse_stats, pulse_tok_stats
lỗi chèn trạng thái/gram, walk quá sâu, tổng gram

pulse_cgroups do loader nạp từ allow-list của resolver; process ngoài allow-list thoát ở bước 1, nên chỉ trả chi phí một lần tra map.

Bố cục token 32 bit (pulse_token_ids.h)

nr syscall (bit 23–31)	loại (21–22)	chi tiết (bit 0–20)

loại 0: chỉ tên · 1: errno (bit 0–11) · 2: bucket kích thước 0–15 · 3: open (bit 20 = ghi, bit 0–19 = FNV thư mục; mã riêng cho "/", "rel", không rõ)
0xFFFFFFFF = ô lịch sử trống (đầu thread → gram ngắn hơn, như TokenLM offline)

Loader (pulse_counter_loader.c, -DPULSE_TOKENS) và đầu ra

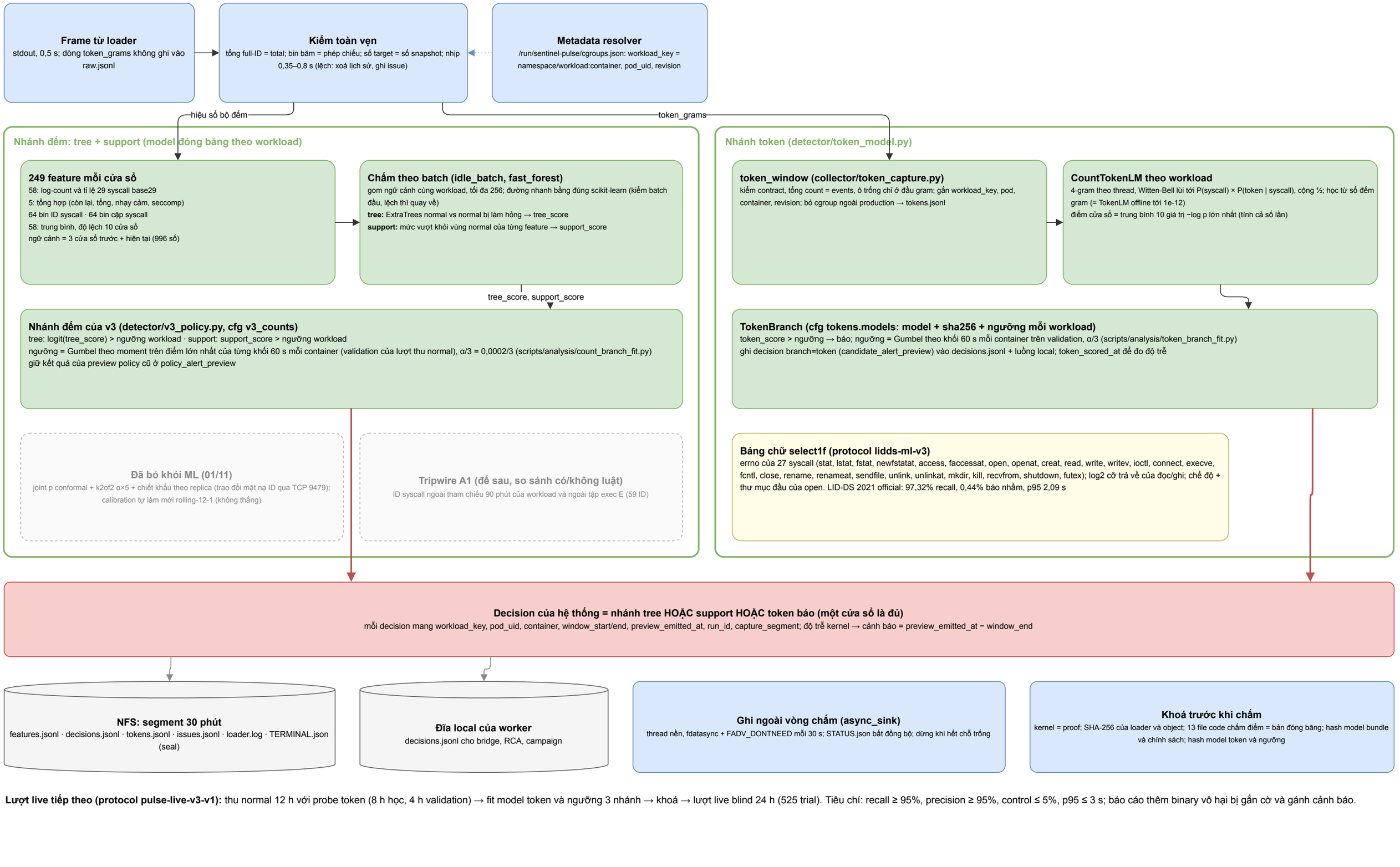
Khởi động
kiểm layout map; đặt .rodata.tokcfg; nạp object; gắn sys_enter, fexit seccomp, sys_exit; nạp allow-list từ resolver; lỗi thì dừng (fail closed)

Mỗi 0,5 s (MONOTONIC + 20 ms)
cộng bộ đếm per-CPU mọi cgroup → cgroup_snapshot_extended; đọc pulse_grams theo lô, lấy cửa sổ đã đóng, xoá theo lô → token_grams

Dòng JSON ra stdout (một frame)
{"type":"cgroup_snapshot_extended","cgroup_id":…,"total":…,"counts":{29 syscall},"syscall_bins":[64],"transition_bins":[64],"full_counts":{…},"seccomp_skipped_or_emulated":…}
{"type":"token_grams","contract":"pulse-token-select1f-v1","cgroup_id":…,"window":W,"window_end":t,"events":n,"grams":[[t0,t1,t2,t3,count],…]}
{"type":"token_drain",…} · {"type":"stat",…} · {"type":"snapshot_end","targets":…,"snapshots":…}
Proof (scripts/token_probe_proof.py): tổng sự kiện token = tổng full_counts trong cgroup cô lập (422.316 = 422.316), token mẫu đúng, seccomp như proof matched.

Worker chấm điểm và lidds-ml-v3 trên cụm

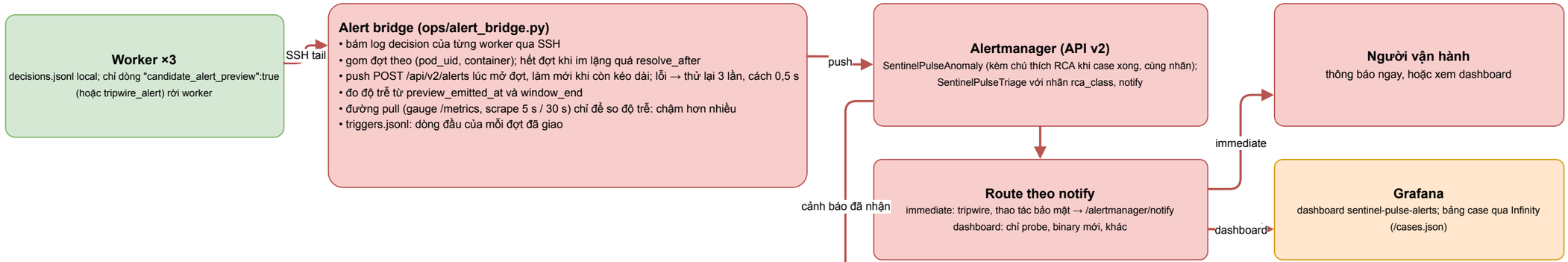
pipeline/matched_worker.py, một unit systemd mỗi worker. Ba nhánh, mỗi nhánh một ngưỡng theo workload; một cửa sổ vượt ngưỡng ở bất kỳ nhánh nào là báo động.



Lượt live tiếp theo (protocol pulse-live-v3-v1): thu normal 12 h với probe token (8 h học, 4 h validation) → fit model token và ngưỡng 3 nhánh → khoá → lượt live blind 24 h (525 trial). Tiêu chí: recall ≥ 95%, precision ≥ 95%, control ≤ 5%, p95 ≤ 3 s; báo cáo thêm binary vô hại bị gắn cờ và gánh cảnh báo.

Đường cảnh báo và truy nguyên nguyên nhân (RCA)

Bridge và RCA chạy trên .234 và chỉ đọc decision. Mã đợt cảnh báo = mã case RCA.



RCA (sentinel_pulse/rca)

3 policy Tetragon (chỉ production)
runtime: file nhạy cảm, credential, kết nối, namespace, ptrace **fork**
flow: lần gửi trên socket đã mở, lấy mẫu 1 lần mỗi process mỗi 10 s
+ exec provenance

sự kiện

Subscriber
tetra getevents của 3 worker, xoay vòng có giới hạn
ghép theo node, Pod UID, container, thời gian; process = exec_id (PID bị tái dùng)
ring SQLite 180 s; nhật ký exec 15 phút

Dựng case (nhịp 0,25 s)
từ cảnh báo Alertmanager đã nhận (triggers.jsonl); lấy 30 s trước cảnh báo
cùng container trong 300 s → cùng case; binary mới trong khoảng đó → case tiếp nối; giữ tối đa 20.000 case

Xếp hạng nghi phạm (heuristic có lý do)
binary mới trong workload sau mốc baseline +4 · thao tác credential / namespace / mount / ptrace +3 mỗi lần (tối đa 9) · binary chưa từng ghi nhận +2 · exec trong cửa sổ cảnh báo +2 · syscall thất bại +1 (tối đa 3) · cha của nghi phạm khác +1
ngoại lệ: hạ quyền setuid/setgid lúc khởi động; containerd-shim, runc không xếp hạng
phá hoà theo thời điểm khởi chạy: top-1 100% trên các trial blind

Triage (ops/rca_view.py)
tripwire → notify ngay · security_operation → notify ngay · probe_only (mọi exec là probe Kubernetes của pod spec) → dashboard · new_binary → dashboard · other → dashboard
chỉ là quyết định giao tin: decision và mọi chỉ số đánh giá không đổi

Đầu ra của case
<case_id>.json (cây bằng chứng) · .md · .svg; trang /cases, /cases/<id> (nghi phạm, lý do, binary, đích mạng, cây)
bridge push lại SentinelPulseAnomaly kèm chú thích RCA (khoảng 2,7 s sau cảnh báo)
cây cho thấy liên quan theo định danh và thời gian, không chứng minh nhân quả; RCA không tự xử lý pod

Lượt hệ thống cuối (25–26/10): 5.840/5.840 đợt tới Alertmanager (3 push lỗi, thử lại thành công); p95 kernel → cảnh báo 1,97 s.

Quy trình chọn, xác nhận và đánh giá

Chọn trên dữ liệu development, xác nhận một lần trên dữ liệu khoá; mỗi bước có protocol đăng ký trước và biên nhận JSON trong repo. Không chạy luật (tripwire, novelty, bỏ phiếu) cho tới khi ML chốt.

LID-DS 2021 development + CB-DS
chọn bảng chữ token, nhánh, ngưỡng; 15 kịch bản; CB-DS 891/891 tấn công, 2/1.410 báo nhầm

→

LID-DS 2021 official
lidds-ml-v3: 97,32% recall (1.237/1.271), 0,44% báo nhầm, precision 97,25%, p95 2,09 s; lần chấm official thứ 2, đã khai báo

Thu normal trên cụm (12 h)
pulse-token-capture: 8 h học model token, 4 h validation cho ngưỡng 3 nhánh; chưa chạy (chờ ablation vòng 2 xong)

Lượt live blind (24 h)
pulse-live-v3: 525 trial, tốc độ chính 6/12/24 mỗi giây; recall, precision, control, p95, binary vô hại, gánh

LID-DS 2019 (cuối cùng)
chấm một lần, protocol lidds2019-confirmation-v1

So sánh có / không luật
tripwire A1 và các luật khác thêm vào ML đã chốt; mở lại baseline học sâu

Offline (máy phân tích .234, systemd-run có giới hạn bộ nhớ)

Chuyển đổi và chấm
sentinel_pulse/lidds: convert.py, signals.py
lidds_ml_token_alphabets.py (token), lidds_ml_scores.py --all-windows (tree, support)
lidds_ml_v2.py --token-column select1f --protocol lidds-ml-v3.json (chọn và chấm)

Ngưỡng
Gumbel theo moment trên điểm lớn nhất mỗi bản ghi validation, mỗi nhánh α/3, α = 0,0002; bằng oracle một mức chung ở báo nhầm 0,6–2%; tốt hơn Gumbel MLE, GEV, normal, đuôi mũ
trên cụm: khối thời gian 60 s mỗi container thay cho bản ghi

Kiểm thêm
ablation syscall theo kiểu bỏ một (vòng 2: 3 seed, 45 cấu hình, Holm)
LSTM lớn không hơn n-gram tổng thể
thứ tự phát token như probe: 530/544 development

Overhead probe token (token-probe-overhead-v1)
benchmark syscall tĩnh, 5 vòng, 3 worker, xoay điều kiện: off / A / B / C / D / D không giám sát
lần 1 fexit → va chạm kprobe Tetragon; lần 2 sys_exit; lần 3 đi ngược dentry; báo cáo, không đạt/trượt

Trên cụm (lượt live)

Planner
plan_matched_run.py --token-build --token-models --v3-counts → CONFIG.json bất biến; launcher cài worker, RCA, bridge, traffic, campaign

Fit sau lượt thu
token_branch_fit.py → MODELS.json (model token + ngưỡng)
count_branch_fit.py → ngưỡng tree, support
khoá sha256 trước lượt live

Chấm lượt live
trial phát hiện khi có báo động trong [start, start + 15 s]; đợt = báo động cùng container cách nhau < 15 s; gánh = đợt / replica-giờ ngoài khoảng campaign; Wilson, bootstrap theo workload

Kết quả hệ thống cuối (ML cũ + tripwire)
pulse-final-system-confirm 25–26/10: recall 99,4%, precision 99,4%, control 0,6%, p95 1,97 s; binary vô hại bị gắn cờ 89,5% → lý do làm ML token

Bằng chứng và tài liệu

Protocol
sentinel_pulse/protocol/*.json, đăng ký trước khi có dữ liệu; amendment ghi rõ lý do

Biên nhận
validation-evidence/results-YYYYMMDD/*: JSON, bảng, log; seal SHA-256 của segment

Audit số liệu
scripts/audit/headline_numbers.py: mọi số trong bản LaTeX khớp biên nhận (48/48)

Đồ án
thesis/ (LaTeX, tectonic); báo cáo và slide họp trong docs/

Ghi chép
notes/STATE.md (trạng thái), notes/WORKLOG.md (nhật ký từng bước)

Nguyên tắc: chọn trên development, official và lượt live chỉ để xác nhận; mỗi lần chấm lại official phải khai báo; không job nặng nào chạy cùng lượt live; số liệu chỉ lấy từ biên nhận.